

Wrapping AWS Lambda ESM handler

The problem: How to transform AWS Lambda handler below?

```
// input.js
export const handler = async(event) => {
  return "Hi from AWS Lambda";
};
```

To the following, notice the WrapAwsLambda wrapper:

```
// transformed.js
export const handler = WrapAwsLambda(async(event) => {
  return "Hi from AWS Lambda";
});
```

Using loader hooks

Built-in Node.js mechanism to hook into the ESM `import` :

<https://nodejs.org/api/module.html#customization-hooks>

Asynchronous hooks

```
// register_async_hooks.mjs
import { register } from "node:module";
register("./async-hooks.mjs", import.meta.url);
```

Add `--import` to register hooks:

```
node --import ./register_async_hooks.mjs runtime.mjs
```

The hooks `load` function runs on a dedicated thread!

Async hooks **load** function

```
// async_hooks.mjs
import { transformLambda } from "../myTransform.js";
let patched = false;
export async function load(url, context, nextLoad) {
  const result = await nextLoad(url, context);
  if (!patched && url.endsWith("/handler.mjs")) {
    patched = true;
    return {
      format: "module", shortCircuit: true,
      source: transformLambda(result.source.toString(), "handler", "WrapAwsLambda")
    };
  }
  return result;
}
```

Note: async hooks are usually 2x slower than sync hooks.

Async hooks - all in one file

Having everything in one file is very convenient:

```
// async-hooks-babel-one-file.mjs
import { register } from "node:module";
register(import.meta.url); //< no extra file!

import { transformLambda } from "../benchmark/lib/babel-transform.js";
export async function load(url, context, nextLoad) { ... }
```

```
node --import ./async-hooks-babel-one-file.mjs runtime.mjs
```

but it adds significant overhead:

- `208.6 ± 6.1 ms` for separate register file + hook file using Babel
- `324.8 ± 5.3 ms` for all in one file using Babel

Synchronous hooks **load** function

```
// sync_hooks.mjs
import { registerHooks } from "node:module";
import { transformLambda } from "../myTransform.js";
let patched = false;
registerHooks({
  load(url, context, nextLoad) {
    const result = nextLoad(url, context);
    if (!patched && url.endsWith("/handler.mjs")) {
      patched = true;
      return {
        format: "module",
        shortCircuit: true,
        source: transformLambda(result.source.toString(), "handler", "WrapAwsLambda")
      };
    }
    return result;
  },
});
```

Transforming ESM source code

Naive approach using `RegExp()` for direct string manipulation:

```
// regex-transform.ts
export function transformLambda(input: string, handler: string, wrapper: string): string {
  return input.replace(
    new RegExp(`export const ${handler} = (.+);`, "s"),
    `export const ${handler} = ${wrapper}($1);`
  );
}
```

This is the fastest way, which we will use as baseline for AST transformations.

Babel transform

Babel transformation with custom `visitor` is used to generate the wrapper call:

```
// babel-transform.ts
ExportNamedDeclaration(path) {
  if (t.isVariableDeclaration(path.node.declaration)) {
    const varDecl = path.node.declaration.declarations[0];
    if (t.isIdentifier(varDecl.id) && varDecl.id.name === handler) {
      path.replaceWith(t.exportNamedDeclaration(
        t.variableDeclaration("const", [t.variableDeclarator(t.identifier(handler),
          t.callExpression(t.identifier(wrapper), [varDecl.init!]))]),
      ));
      path.skip();
    }
  }
}
```

Acorn => estraverse => astring

Acorn is only a parser, `estraverse` and `astring` adds traversing and codegen.

```
const ast = acorn.parse(code, { ecmaVersion: 2020, sourceType: "module" });
estraverse.replace(ast as ESTree.Node, {
  enter: function (node) {
    if (node.type === "ExportNamedDeclaration"
        && node.declaration?.type === "VariableDeclaration") {
      const varDecl = node.declaration.declarations[0];
      if (varDecl.id.type === "Identifier" && varDecl.id.name === handler) {
        varDecl.init = NESTree.CallExpression(Helpers.AutoChain(wrapper),
            [varDecl.init as NESTree.Expression]) as ESTree.Expression;
      }
      return { ...node };
    }
  }
});
return astring.generate(ast);
```


Rust native addon using `oxc.rs`

The `oxc.rs` tools and crate `oxc_transformer` provides API similar to Babel in Rust.

With `napi.rs` we can easily export the Rust code:

```
use napi_derive::napi;
mod transform;

#[napi]
pub fn transform_lambda(input: String) -> String {
    transform::transform_lambda_source(input)
}
```

```
const output = transformLambda(input);
```

Rust wasm plugin for `swc.rs`

Based on the [SWC](#) guide for [plugin creation](#) compiled to WebAssembly.

```
// swc-wrapper.cjs
const swc = require("@swc/core");
exports.transformLambda = function (sourceCode, handler, wrapper) {
  const output = swc.transformSync(sourceCode, {
    filename: "handler.mjs", sourceMaps: false, isModule: true, {
      target: "esnext", experimental: {
        plugins: [[ require.resolve("./my-plugin.wasm"), { handler, wrapper } ]]
      }
    },
  });
  return output.code;
};
```

LD_PRELOAD - `open()` detour

Detour of `open()` from `libc` allows us to redirect to temporary file or intercept the file descriptor and provide `read()` and `uv_fs_fstat()` detours for reading and file size.

This can be achieved using [Frida](#) and the [Rust bindings example](#).

The resulting library is preloaded before starting the Node process.

```
LD_PRELOAD=../wrap-esm-lambda.linux-x64-gnu.node node runtime.mjs
```

This is a little bit faster (1-2 ms) than using `require` + calling exported function:

```
const { installHooks } = require("../wrap-esm-lambda.linux-x64-gnu.node");  
installHooks(); //< detours for `open()`, `read()` and `uv_fs_fstat()`
```

Benchmark

Benchmark table via [hyperfine](#) and `usr/bin/time -v` for Max RSS:

Hook	Mean [ms]	Min [ms]	Max [ms]	Relative	Max RSS [MB]
regex	24.3 ± 0.9	22.8	28.6	1.03 ± 0.06	44.24
LD_PRELOAD	25.8 ± 0.7	24.4	28.2	1.10 ± 0.06	49.26
oxc	35.6 ± 1.0	33.9	38.4	1.51 ± 0.09	54.92
acorn	45.0 ± 1.4	43.2	50.5	1.91 ± 0.11	56.30
swc plugin	127.4 ± 4.3	120.8	135.1	5.42 ± 0.33	371.61
babel	180.0 ± 4.0	172.1	188.2	7.66 ± 0.42	82.51
async babel	211.4 ± 4.2	205.9	220.3	9.00 ± 0.49	90.44

